

Trie Ez Version :

```

struct Trie {
    vector<Node> tr;

    int newNode() {
        tr.emplace_back();
        return tr.size() - 1;
    }

    Trie() { tr.clear(), newNode(); }

    void insert(const string &s) {
        int u = 0;
        for (char c: s) {
            if (!tr[u][c])
                tr[u][c] = newNode();
            tr[u].sz++;
            u = tr[u][c];
        }
        tr[u].sz++;
        tr[u].isEnd++;
    }

    int getSize(string &s){
        int u = 0;
        for (char c: s) {
            if (!tr[u][c])
                return 0;
            u = tr[u][c];
        }
        return tr[u].sz;
    }
};

```

Trie

```

int g = 0 ;
struct Node
{
    map<char,int>mp ;
    int sz = 0 , isEnd = 0;
    pair<int,int>mx = {0 , 0} ;
    int &operator[](char x){
        return mp[x];
    }
}

```

```

};
struct Trie
{
    vector<Node>tr ;
    int newNode (){
        tr.emplace_back();
        return tr.size()-1 ;
    }
    Trie(){
        tr.clear() ;
        newNode() ;
    }
    void insert(string &s , int i = 0 , int cur = 0 ){
        if(i==s.size()){
            tr[cur].isEnd ++;
            tr[cur].mx = min(tr[cur].mx , {-tr[cur].isEnd , -1});
            return ;
        }
        if(!tr[cur][s[i]]){
            tr[cur][s[i]] = newNode() ;
        }
        int newcur = tr[cur][s[i]];
        tr[cur].sz++;
        insert(s,i+1,newcur);
        tr[cur].mx = min(tr[cur].mx , {tr[newcur].mx.first , s[i]});
    }

    string search (string s){

        int cur = 0 ;
        for(auto ch : s){
            if(tr[cur][ch]==0){
                return "" ;
            }
            cur = tr[cur][ch] ;
        }
        string res = s ;
        int f = 0;
        while (f!=-1)
        {
            f = tr[cur].mx.second ;
            if(f==-1)break;
            cur = tr[cur][f] ;
            res+=f ;
        }
        g = -tr[cur].mx.first ;
        return res ;
    }
};

```

Binary Trie

```
int M = 60 ;

struct Node
{
    int mp[2] = {0,0} ;
    int sz = 0 ;
    int isEnd = 0 ;
    int &operator[](int x){
        return mp[x] ;
    };
};

struct Trie
{
    vector<Node>tr ;
    int newNode(){
        tr.emplace_back() ;
        return tr.size()-1 ;
    }
    Trie(){
        tr.clear() ;
        newNode();
    }
    void insert(int x , int op){
        int cur = 0 ;
        for (int i = M-1; i >= 0; i--)
        {
            int u = x>>i & 111 ;
            if(tr[cur][u]==0){
                tr[cur][u] = newNode();
            }
            cur = tr[cur][u] ;
            tr[cur].sz+=op ;
        }
        tr[cur].isEnd+=op ;
    }
    // min val of a[i] ^ x
    int queryMin(int x){
        int cur = 0 ; int res = 0 ;
        for (int i = M-1; i >= 0; i--)
        {
            int u = x>>i & 111 ;
            int v = tr[cur][u];
            if(!tr[v].sz){
                res+=(111<<i) ;
                cur = tr[cur][!u];
            }else{
                cur = v ;
            }
        }
        return res ;
    }
}
```

```

// max val of a[i] ^ x
int queryMax(int x) {
    int cur = 0 ; int res = 0 ;
    for (int i = M - 1; i >= 0; --i) {
        int u = x >> i & 111;
        int v = tr[cur][!u];
        if(tr[v].sz){
            res+=(111<<i) ;
            cur = v;
        }else{
            cur = tr[cur][u] ;
        }
    }
    return res ;
}

// count how many times x is in the trie
int count(int x){
    int cur = 0 ;
    for (int i = M-1; i >= 0; i--){
        int u = x>>i & 111 ;
        if(tr[cur][u]==0) return 0 ;
        cur = tr[cur][u] ;
    }
    return tr[cur].isEnd ;
}

// count how many numbers in trie are < x
int countLess(int x){
    int cur = 0 ; int res = 0 ;
    for (int i = M-1; i >= 0; i--){
        int u = x>>i & 111 ;
        if(u==1 && tr[cur][0]) res += tr[tr[cur][0]].sz ;
        if(tr[cur][u]==0) break ;
        cur = tr[cur][u] ;
    }
    return res ;
}

// count number of a[i]^num < K
int query(int num, int K){
    int cur = 0 , res = 0 ;
    for(int i = M - 1 ; i >= 0 ; i--){
        int n = num >> i & 111 ;
        int k = K >> i & 111 ;
        if(n == 0 && k == 0){
            cur = tr[cur][0] ;
        }else if(n == 0 && k == 1){
            if(tr[cur][0]) res += tr[tr[cur][0]].sz ;
            cur = tr[cur][1] ;
        }else if(n == 1 && k == 0){
            cur = tr[cur][1] ;
        }else{ // n == 1 && k == 1
            if(tr[cur][1]) res += tr[tr[cur][1]].sz ;
            cur = tr[cur][0] ;
        }
    }
    if(cur == 0) break ;
}

```

```

    }
    if(cur) res += tr[cur].isEnd ;
    return res ;
}

int kth(int k){
    int cur = 0, res = 0;
    for(int i = M - 1; i >= 0; i--){
        int left = tr[cur][0];
        int szLeft = (left ? tr[left].sz : 0);
        if(k <= szLeft){
            cur = left;
        }else{
            k -= szLeft;
            res |= (1ll << i);
            cur = tr[cur][1];
        }
        if(cur == 0) break;
    }
    return res;
}

};

```

Hashing

```

const int M = 2 , N = 1e6 + 5;
array<int,2> B , MODS , pw [N];
void build() {
    if (B[0]) return;

    random_device rd;
    mt19937 mt(rd());

    auto rnd = [&](int l, int r) {
        return uniform_int_distribution<int>(l, r)(mt);
    };

    auto is_prime = [](int x) {
        if (x < 2) return false;
        for (int i = 2; i * i <= x; ++i)
            if (x % i == 0) return false;
        return true;
    };

    B[0] = rnd(100, 500);
    B[1] = rnd(100, 500);

    MODS[0] = rnd(2e8, 2e9);

```

```

MODS[1] = rnd(2e8, 2e9);

while (!is_prime(MODS[0])) MODS[0]--;
while (!is_prime(MODS[1]) || MODS[0] == MODS[1]) MODS[1]--;

for (int j = 0; j < M; ++j) {
    pw[0][j] = 1;
    for (int i = 1; i < N; ++i) {
        pw[i][j] = (1LL * pw[i-1][j] * B[j]) % MODS[j];
    }
}

}

struct Hash
{
    vector<array<int,M>>pref , rev ;

    void initPref (string &s){
        int n = s.size() ;
        pref.assign(n,{}) ;
        array<int,2> h {} ;
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < M; j++)
            {
                h[j] = (h[j] * B[j]) % MODS[j];
                h[j] = (h[j] + s[i]) % MODS[j];
            }
            pref[i] = h ;
        }
    }

    void initSuff (string &s){
        int n = s.size() ;
        rev.assign(n,{}) ;
        array<int,2> h {} ;
        for (int i = n-1; i >= 0; i++)
        {
            for (int j = 0; j < M; j++)
            {
                h[j] = (h[j] * B[j]) % MODS[j];
                h[j] = (h[j] + s[i]) % MODS[j];
            }
            rev[i] = h ;
        }
    }

    array<int , M> substr (int l , int r){
        if(!l) return pref[r] ;
        auto res = pref[r] ;
        for (int j = 0; j < M; j++)
        {
            res[j] -= (1LL * pw[r-l+1][j] * pref[l-1][j]) % MODS[j] ;
            if(res[j] < 0) res[j]+= MODS[j] ;
        }
        return res ;
    }
}

```

```

}
array<int , M> substrRev (int l , int r){
    if(r == rev.size()-1) return rev[l] ;
    auto res = rev[l] ;
    for (int j = 0; j < M; j++)
    {
        res[j] -= (1LL * pw[r-l+1][j] * rev[l-1][j]) % MODS[j] ;
        if(res[j] < 0) res[j]+= MODS[j] ;
    }
    return res ;
}
bool isPal (int l , int r){
    return substr(l , r) == substrRev(l , r) ;
}
};

```

manacher Longest plaindromic substring in O(n) time complexity

```

// Longest plaindromic substring in O(n) time complexity
string manacher(string s) {
    string arr;
    for (char c : s) {
        arr.push_back('#');
        arr.push_back(c);
    }
    arr.push_back('#');

    int n = arr.size();
    vector<int> dp(n, 0);
    int center = 0, right = 0;
    int maxLen = 0, maxCenter = 0;

    for (int i = 0; i < n; ++i) {
        int mirror = 2 * center - i;
        if (i < right)
            dp[i] = min(right - i, dp[mirror]);

        int a = i + (dp[i] + 1);
        int b = i - (dp[i] + 1);

        while (a < n && b >= 0 && arr[a] == arr[b]) {
            dp[i]++;
            a++;
            b--;
        }

        if (i + dp[i] > right) {

```

```
        center = i;
        right = i + dp[i];
    }

    if (dp[i] > maxLen) {
        maxLen = dp[i];
        maxCenter = i;
    }
}

string res = "";
for (int i = maxCenter - maxLen; i <= maxCenter + maxLen; ++i) {
    if (arr[i] != '#')
        res += arr[i];
}
return res;
}
```
